

MongoDB: foodieApp Database

Each question is shown in the blue box, followed by its answer. Note: the older shell was called mongo, and current MongoDB versions use mongosh. The commands below work the same in both.

Question 1

Question 1. *Install MongoDB Community Edition on your system and use the mongo shell to create a new database named 'foodieApp'.*

Answer:

Step 1: Install MongoDB Community Edition

Windows

1. Download the MongoDB Community Server MSI installer from mongodb.com/try/download/community.
2. Run the installer, choose the **Complete** setup, and keep **Install MongoDB as a Service** ticked so it starts automatically.
3. Optionally install MongoDB Compass (the GUI tool) when the installer offers it.
4. If the shell is not included, download **MongoDB Shell (mongosh)** from mongodb.com/try/download/shell and install it.

macOS (Homebrew)

```
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community
```

Ubuntu/Linux: add the official MongoDB repository and install the mongodb-org package by following the guide for your Ubuntu version at mongodb.com/docs/manual/installation, then start the service:

```
sudo systemctl start mongod
sudo systemctl status mongod
```

Step 2: Verify the installation

```
mongod --version
mongosh --version
```

Step 3: Open the shell and create the database

```
mongosh
```

This connects to the local server at `mongodb://127.0.0.1:27017`. Then run:

```
use foodieApp
```

Output:

```
switched to db foodieApp
```

Important: MongoDB creates a database only when the first data is stored in it. Right after use foodieApp, the database will not yet appear in `show dbs`. It appears after you insert the first document in Question 2.

Check the current database with:

```
db
```

```
foodieApp
```

Question 2

Question 2. Inside the 'foodieApp' database, create a collection named 'restaurants' and insert a document for a Zomato-style restaurant with fields: name, cuisine, rating, and location.

Answer: a collection is created automatically when you insert the first document, so `insertOne()` is enough. You can also create it explicitly with `db.createCollection("restaurants")`.

```
use foodieApp

db.restaurants.insertOne({
  name: "Spice Villa",
  cuisine: "North Indian",
  rating: 4.5,
  location: { city: "Mumbai", area: "Bandra" }
})
```

Output (your ObjectId will be different):

```
{
  acknowledged: true,
  insertedId: ObjectId('66f2a1c3e4b0a1b2c3d4e5f6')
}
```

Confirm the collection and database now exist:

```
show collections
show dbs
```

```
restaurants

admin      40.00 KiB
config     72.00 KiB
foodieApp  40.00 KiB
local      72.00 KiB
```

The sizes will differ on your system. The important point is that `foodieApp` is now listed.

Question 3

Question 3. Insert three more restaurant documents into the 'restaurants' collection, making sure to use at least one field with an array data type (for example, a 'menu' array listing popular dishes).

Answer: use `insertMany()` to add all three at once. Each document has a menu array of dish names.

```
db.restaurants.insertMany([
  {
    name: "Pizza Corner",
    cuisine: "Italian",
    rating: 4.2,
    location: { city: "Delhi", area: "Connaught Place" },
    menu: ["Margherita Pizza", "Garlic Bread", "Pasta Alfredo"]
  },
  {
    name: "Dragon Wok",
    cuisine: "Chinese",
    rating: 4.1,
    location: { city: "Bengaluru", area: "Indiranagar" },
    menu: ["Hakka Noodles", "Manchurian", "Spring Rolls"]
  },
  {
    name: "South Spice",
    cuisine: "South Indian",
    rating: 4.7,
    location: { city: "Chennai", area: "T Nagar" },
    menu: ["Masala Dosa", "Idli Sambar", "Filter Coffee"]
  }
])
```

Output (ObjectIds will differ):

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('66f2a1d0e4b0a1b2c3d4e5f7'),
    '1': ObjectId('66f2a1d0e4b0a1b2c3d4e5f8'),
    '2': ObjectId('66f2a1d0e4b0a1b2c3d4e5f9')
  }
}
```

Array field used: menu in all three new documents. Count the documents to check:

```
db.restaurants.countDocuments()
```

4

Question 4

Question 4. Retrieve all documents from the 'restaurants' collection and display them in the mongo shell. Identify which BSON types are used for each field. Hint: Look for String, Number, Array, and Object types in your documents.

Answer:

Retrieve all documents

```
db.restaurants.find()
```

In mongosh the output is already nicely formatted. In the older mongo shell, use `db.restaurants.find().pretty()`.

Output (ObjectIds will differ):

```
[
  {
    _id: ObjectId('66f2a1c3e4b0a1b2c3d4e5f6'),
    name: 'Spice Villa',
    cuisine: 'North Indian',
    rating: 4.5,
    location: { city: 'Mumbai', area: 'Bandra' }
  },
  {
    _id: ObjectId('66f2a1d0e4b0a1b2c3d4e5f7'),
    name: 'Pizza Corner',
    cuisine: 'Italian',
    rating: 4.2,
    location: { city: 'Delhi', area: 'Connaught Place' },
    menu: [ 'Margherita Pizza', 'Garlic Bread', 'Pasta Alfredo' ]
  },
  {
    _id: ObjectId('66f2a1d0e4b0a1b2c3d4e5f8'),
    name: 'Dragon Wok',
    cuisine: 'Chinese',
    rating: 4.1,
    location: { city: 'Bengaluru', area: 'Indiranagar' },
    menu: [ 'Hakka Noodles', 'Manchurian', 'Spring Rolls' ]
  },
  {
    _id: ObjectId('66f2a1d0e4b0a1b2c3d4e5f9'),
    name: 'South Spice',
    cuisine: 'South Indian',
    rating: 4.7,
    location: { city: 'Chennai', area: 'T Nagar' },
    menu: [ 'Masala Dosa', 'Idli Sambar', 'Filter Coffee' ]
  }
]
```

BSON types used for each field

Field	Example value	BSON type
-------	---------------	-----------

_id	ObjectId('66f2a1c3...')	ObjectId (added automatically)
name	"Spice Villa"	String
cuisine	"North Indian"	String
rating	4.5	Number (stored as Double)
location	{ city: "Mumbai", area: "Bandra" }	Object (embedded document)
location.city	"Mumbai"	String (inside the Object)
location.area	"Bandra"	String (inside the Object)
menu	["Masala Dosa", "Idli Sambar", ...]	Array (each item is a String)

Note: the menu field exists only in the last three documents. The first document (Spice Villa) has no menu, because MongoDB documents in one collection do not all need the same fields.

Checking the types in the shell

Use the \$type operator to let MongoDB report the type of each field:

```
db.restaurants.aggregate([
  { $project: {
    name: { $type: "$name" },
    cuisine: { $type: "$cuisine" },
    rating: { $type: "$rating" },
    location: { $type: "$location" },
    menu: { $type: "$menu" }
  } }
])
```

Output for the second document (Pizza Corner):

```
{
  _id: ObjectId('66f2a1d0e4b0a1b2c3d4e5f7'),
  name: 'string',
  cuisine: 'string',
  rating: 'double',
  location: 'object',
  menu: 'array'
}
```

For the first document, the menu field shows 'missing' because that field does not exist there.

You can also search by type, for example to find all documents whose menu is an array:

```
db.restaurants.find({ menu: { $type: "array" } })
```

This returns Pizza Corner, Dragon Wok and South Spice.